Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «АПО ЭВМ»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – обработка символьных данных»

Выполнил:
Студент 4 курса
Группы АС-66
Лысюк Р. А.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, практическая работа с подсистемой регистровой и оперативной памяти.

Ход работы

1. *Постановка задачи.* Дан текст – непустая последовательность не длиннее ста символов. Признаком конца ввода является точка, в сам текст точка не входит.

Проверить, удовлетворяет ли текст заданному условию. Если условие выполнено, преобразовать текст по указанному в варианте правилу; в противном случае выдать соответствующее сообщение. Преобразованный текст напечатать.

Проверяемое условие и правила обработки текста определяются конкретным вариантом задания.

Если введенная последовательность символов не является текстом, преобразовывать ее не нужно, а следует напечатать соответствующее сообщение.

Ввод текста, проверка условия, обработка текста и печать результата должны выполняться последовательно, отдельными частями программы.

2. Используя знания и навыки, полученные в результате выполнения лабораторной работы №1, а также изучив дополнительные теоретические материалы, приведенные в методическом указании, написать программу на ассемблере, реализующую обработку текста согласно варианту, выданному преподавателем.

Содержание отчета:

- титульный лист;
- описание варианта задания;
- текст программы;
- скриншоты результатов.

Вариант 4: Текст содержит не менее трех латинских букв. Заменить каждую прописную латинскую букву следующей за ней по алфавиту, букву Z менять на A.

Текст программы:

```
.model small

.stack 100h


MAX_LEN equ 100


.data

   msg_hello   db 'Lab 2. Variant 4.', 13,10
               db 'Uslovie: text soderzhit ne menee 3 latinskih bukv.', 13,10
               db 'Preobrazovanie: propisnaya latinskaya bukva ->', 13,10
               db 'sleduyushaya po alfavitu (Z -> A).', 13,10, '$'
   msg_prompt  db 13,10, 'Vvedite text (konec - tochka): $'
   msg_ok      db 13,10, 'Uslovie VYPOLNENO.', 13,10
               db 'Preobrazovannyj text:', 13,10, '$'
   msg_fail    db 13,10, 'Uslovie NE vypolneno: latinskih bukv < 3.', 13,10, '$'
```

```
    msg_crlf    db 13,10, '$'
    msg_empty   db 13,10, 'Text pust.', 13,10, '$'

    in_max      db MAX_LEN+1
    in_act      db ?
    in_buf      db MAX_LEN+2 dup(?)

.code
start:
    mov ax, @data
    mov ds, ax

    lea dx, msg_hello
    mov ah, 09h
    int 21h

    lea dx, msg_prompt
    mov ah, 09h
    int 21h

    lea dx, in_max
    mov ah, 0Ah
    int 21h

    call prepare_text
    jc  no_dot_error

    test cx, cx
    jnz  not_empty
    lea dx, msg_empty
    mov ah, 09h
    int 21h
    jmp  finish

not_empty:
    call check_condition
    test al, al
    jz  cond_fail

    lea dx, msg_ok
```

```
    mov ah, 09h
    int 21h

    call get_text_ptr
    call transform

    call get_text_ptr
    call print_text
    jmp  finish

cond_fail:
    lea dx, msg_fail
    mov ah, 09h
    int 21h
    jmp  finish

no_dot_error:
    lea dx, msg_crlf
    mov ah, 09h
    int 21h
    lea dx, msg_fail
    mov ah, 09h
    int 21h

finish:
    mov ax, 4C00h
    int 21h

prepare_text proc
    push ax
    push bx
    push di

    xor cx, cx
    mov cl, in_act
    test cx, cx
    jz  pt_fail

    lea si, in_buf
    mov di, si
```

```
pt_scan:
    cmp byte ptr [di], '.'
    je  pt_found
    inc di
    loop pt_scan

pt_fail:
    stc
    jmp pt_ret

pt_found:
    mov byte ptr [di], '$'
    mov cx, di
    sub cx, si
    clc
pt_ret:
    pop di
    pop bx
    pop ax
    ret
prepare_text endp

get_text_ptr proc
    push ax
    push di

    lea si, in_buf
    mov di, si
gt_scan:
    cmp byte ptr [di], '$'
    je  gt_done
    inc di
    jmp gt_scan
gt_done:
    mov cx, di
    sub cx, si

    pop di
    pop ax
```

```
    ret
get_text_ptr endp

check_condition proc
    push si
    push cx
    push bx

    xor bx, bx

cc_loop:
    test cx, cx
    jz  cc_finish
    lodsb
    call is_latin
    jnz cc_skip
    inc bx
    cmp bx, 3
    jae cc_true
cc_skip:
    dec cx
    jmp cc_loop

cc_finish:
    xor al, al
    jmp cc_ret

cc_true:
    mov al, 1
cc_ret:
    pop bx
    pop cx
    pop si
    ret
check_condition endp

is_latin proc
    push ax
    cmp al, 'A'
    jb  il_no
```

```
    cmp al, 'Z'
    jbe il_yes
    cmp al, 'a'
    jb  il_no
    cmp al, 'z'
    jbe il_yes
il_no:
    pop ax
    or  al, 1
    ret
il_yes:
    pop ax
    xor al, al
    ret
is_latin endp

transform proc
    push si
    push cx
    push ax

tr_loop:
    test cx, cx
    jz  tr_done

    mov al, [si]

    cmp al, 'A'
    jb  tr_next
    cmp al, 'Z'
    ja  tr_next

    cmp al, 'Z'
    jne tr_inc
    mov al, 'A'
    jmp tr_store
tr_inc:
    inc al
tr_store:
    mov [si], al
```

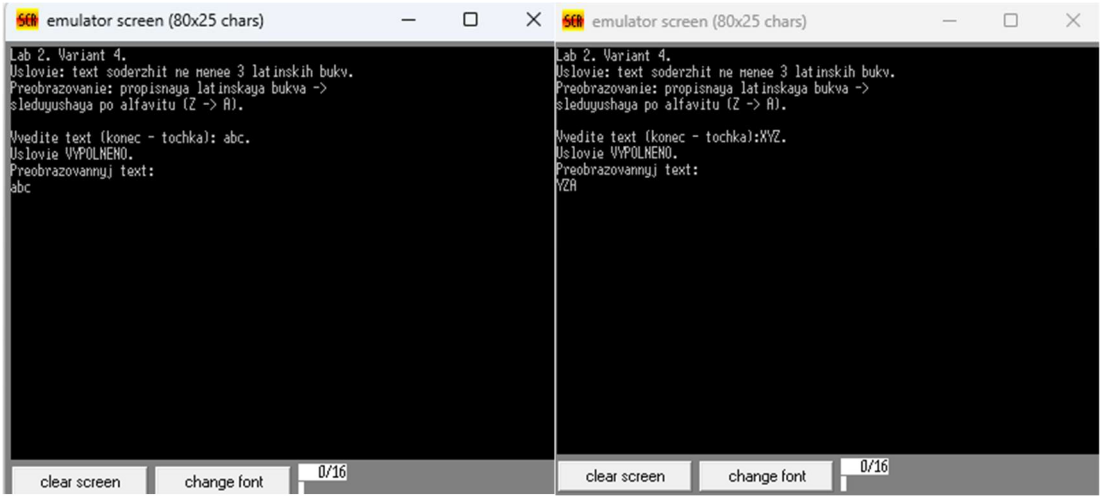
```
tr_next:
    inc si
    dec cx
    jmp tr_loop

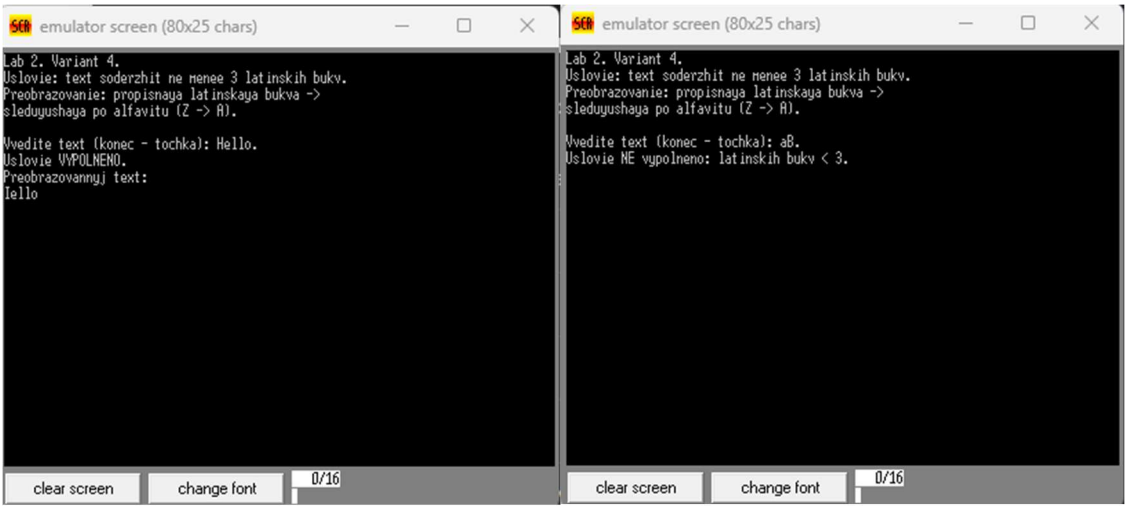
tr_done:
    pop ax
    pop cx
    pop si
    ret
transform endp

print_text proc
    push ax
    push dx
    mov dx, si
    mov ah, 09h
    int 21h
    lea dx, msg_crlf
    mov ah, 09h
    int 21h
    pop dx
    pop ax
    ret
print_text endp

end start
```

Результат работы программы:





Вывод: В ходе лабораторной работы изучена архитектура процессоров Intel и приобретены навыки работы с регистровой и оперативной памятью. Разработана программа на ассемблере по варианту 4: проверка условия «не менее трёх латинских букв» и замена каждой прописной латинской буквы следующей по алфавиту (Z → A). Программа реализована отдельными процедурами без глобальных переменных и без дополнительной памяти. Корректность подтверждена тестами: XYZ. → YZA, Hello. → Iello, aB. → «условие не выполнено».